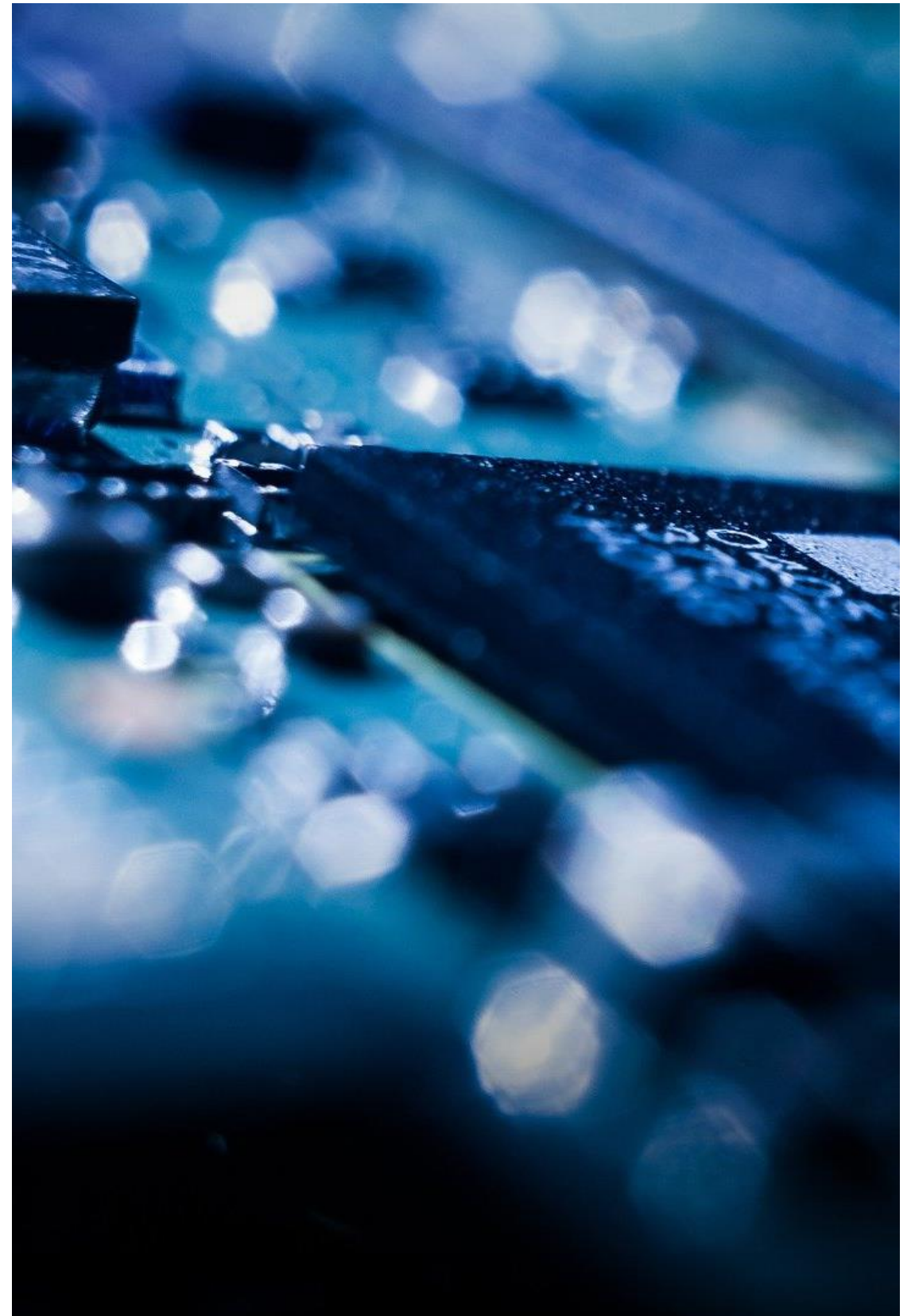


Programmierung eingebetteter Systeme

Standardausgabe und
Variablenerzeugung

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



- ▶ Ausgabe mit `printf` (Allgemein und Formatierung)
- ▶ Ausgabe mit `printf` (Character-Felder)
- ▶ Erzeugung von Variablen

Mit Hilfe der **#include**-Anweisung (an den Präprozessor) ist es möglich, eine externe **Definitionsdatei** (*header file*) für die Dauer des Übersetzungslaufes in die eigene Quelldatei zu kopieren.

Eine Definitionsdatei kann z.B. Deklarationen von Variablen oder Bibliotheksfunktionen enthalten.

Beispiele ANSI-Library:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
...
```

Einfaches Programmbeispiel

```
#include <stdio.h>
```

Bibliotheksfunktionen (ANSI-Library)

```
// --- constants -----
```

```
#define VALUE 5
```

```
// --- prototypes -----
```

```
void main(void);
```

```
// --- modul-variables -----
```

```
int count;
```

```
int dummy;
```

```
// --- start of program -----
```

```
void main(void)
```

```
{
```

```
    count = 0;
```

```
    for (;;) 
```

```
    {
```

```
        count = count + VALUE;
```

```
        printf("test %i", count);
```

Ausgabe auf Standardschnittstelle

```
    }
```

```
}
```

```
// --- end of main -----
```

enthält unter anderem

```
printf (" ... ", v1, v2, ... );
```

Text einschließlich
Sonderzeichen und
Platzhalter für
Variableninhalte

Liste von Variablen

Beispiel:

```
int count;  
float wert;  
...
```

```
printf ("Zaehler: %i Analogwert %f \n\r", count, wert);
```

Erster Platzhalter für erste Variable in der Liste

```
printf (" ... ", v1, v2, ... );
```

Text einschließlich
Sonderzeichen und
Platzhalter für
Variableninhalte

Liste von Variablen

Beispiel:

```
int count;  
float wert;  
...
```

```
printf ("Zaehler: %i Analogwert %f \n\r", count, wert);
```

Zweiter Platzhalter für zweite Variable in der Liste

```
printf (" ... ", v1, v2, ... );
```

Text einschließlich
Sonderzeichen und
Platzhalter für
Variableninhalte

Liste von Variablen

Beispiel:

```
int count;  
float wert;  
...
```

```
printf ("Zaehler: %i Analogwert %f \n\r", count, wert);
```

Sonderzeichen 'new line' und 'return'

Sonderzeichen (Steuerzeichen) in der Standardausgabe `printf`

<code>\n</code>	line feed, Zeilenvorschub	(ASCII-Code: 0x0A)
<code>\r</code>	carriage return, Wagenrücklauf	(ASCII-Code: 0x0D)
<code>\0</code>	String-Ende	(ASCII-Code: 0x00)

<code>\b</code>	backspace
<code>\f</code>	form feed, Seitenvorschub
<code>\t</code>	horizontaler Tab
<code>\v</code>	vertikaler Tab
<code>\\</code>	backslash
<code>\"</code>	Anführungszeichen (doppelt)
<code>\'</code>	Anführungszeichen (einfach)

Definition von Character-Variablen

```
char pe_char;                char pe_char = '?';  
...  
pe_char = '?';
```

Ausgabe-Formatelement für Character-Variablen

%

c

char-Variable

```
printf ("Zeichen : %c\n\r",pe_char);  
printf ("%c-Zeichen\n\r",pe_char);
```

Zeichen : ?

?-Zeichen

Definition von Integer-Variablen

```
int pe_int;                int pe_int = 25;
...
pe_int = 25;
```

Ausgabe-Formatelement für Integer-Variablen in dezimaler Darstellung

%	
-	(optional) linksbündig
+	(optional) auch positives Vorzeichen darstellen
<i>b</i>	(optional) Minimalbreite des Ausgabefeldes
<i>.n</i>	(optional) Mindestzahl der ausgegebenen Ziffern
d	int-Variable

```
printf (>%d<Punkte\n\r", pe_int);
```

```
printf ("%5d\n\r", pe_int);
```

>25<Punkte

25

Definition von Integer-Variablen

```
int pe_int;                int pe_int = 10;
...
pe_int = 10;
```

Ausgabe-Formatelement für Integer-Variablen in hexadezimaler Darstellung

%

-	(optional) linksbündig
<i>b</i>	(optional) Minimalbreite des Ausgabefeldes
<i>.n</i>	(optional) Mindestzahl der ausgegebenen Ziffern
X	int-Variable in hexadezimaler Darstellung (0-9, A-F)

```
printf ("Zaehler:%X\n\r",pe_int);
```

```
printf ("%4.4X\n\r",pe_int);
```

Zaehler:A

000A

Definition von Long Integer -Variablen

```
long int pe_int;                long int pe_int = 50123123;
...
pe_int = 50123123;
```

Ausgabe-Formatelement für Integer-Variablen in dezimaler Darstellung

%

-	(optional) linksbündig
+	(optional) auch positives Vorzeichen darstellen
<i>b</i>	(optional) Minimalbreite des Ausgabefeldes
<i>.n</i>	(optional) Mindestzahl der ausgegebenen Ziffern
ld	int-Variable

```
printf ("z:%ld\n\r",pe_int);
```

```
printf ("%+12.12ld\n\r",pe_int);
```

z:50123123

+000050123123

Definition von Float -Variablen (Gleitkommazahlen)

```
float pe_f1;                                float pe_f1 = 3.1415927;  
...  
pe_f1 = 3.1415927;
```

Ausgabe-Formatelement für Float-Variablen in dezimaler Darstellung

%	
-	(optional) linksbündig
+	(optional) auch positives Vorzeichen darstellen
b	(optional) Minimalbreite des Ausgabefeldes
.n	(optional) Ziffern nach dem Dezimalpunkt
f	float-Variable (Darstellung: ddd.ddddd)
e	float-Variable (Darstellung: d.dddddedd)

```
printf ("pi:%f\n\r",pe_f1);
```

```
printf ("pi(kurz):%.2f\n\r",pe_f1);
```

```
pi:3.1415927
```

```
pi(kurz):3.14
```

Ausgabe

Character-Felder

Einfaches Programmbeispiel

```
#include <stdio.h>
// --- constants -----
#define VALUE 5
// --- prototypes -----
void main(void);
// --- modul-variables -----
int count;
char xy[10];
// --- start of program -----
void main(void)
{
    xy[0] = 't';
    xy[1] = 'e';
    ...
    for (;;)
    {
        ...
    }
}
// --- end of main -----
```

Bibliotheksfunktionen

Definition von Konstanten

Bekanntgabe von Funktionen und Variablentypen

Erzeugen von Variablen
(Speicherplatz bereitstellen)

„Hauptprogramm“ (erste Funktion)

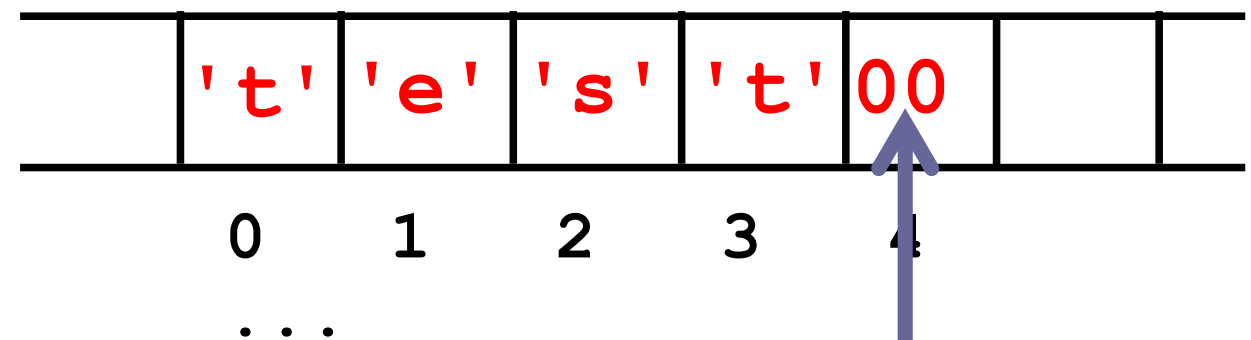
Endlos-Schleife (kein Betriebssystem)

Definition von Character-Feldern („String-Variablen“)

```
char xy[10];                char xy[] = "test";  
...  
strcpy(&xy[0], "test");
```

Felder: Variablen gleichen Typs können über einen Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden.

```
xy[0] = 't';  
xy[1] = 'e';  
xy[2] = 's';  
xy[3] = 't';  
xy[4] = 0x00;
```



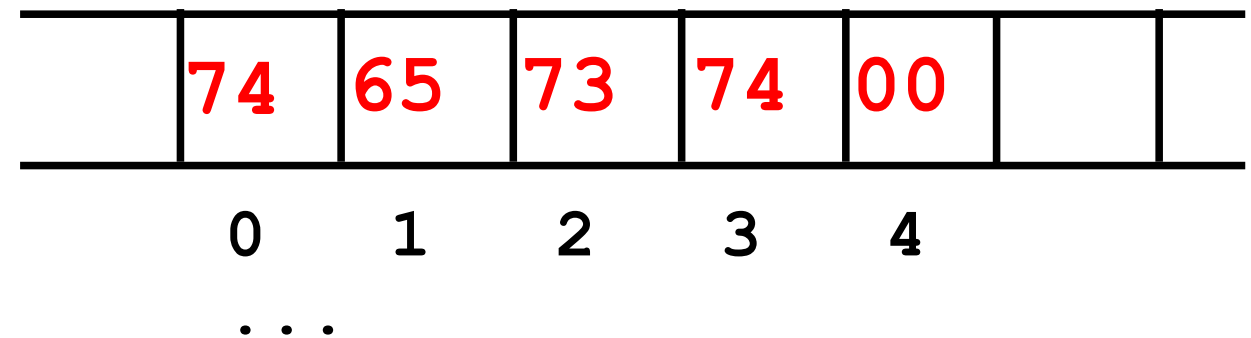
Endekennung !!!

Definition von Character-Feldern („String-Variablen“)

```
char xy[10];           char xy[] = "test";  
...  
strcpy(&xy[0], "test");
```

Felder: Variablen gleichen Typs können über einen Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden.

```
xy[0] = 't';  
xy[1] = 'e';  
xy[2] = 's';  
xy[3] = 't';  
xy[4] = 0x00;
```

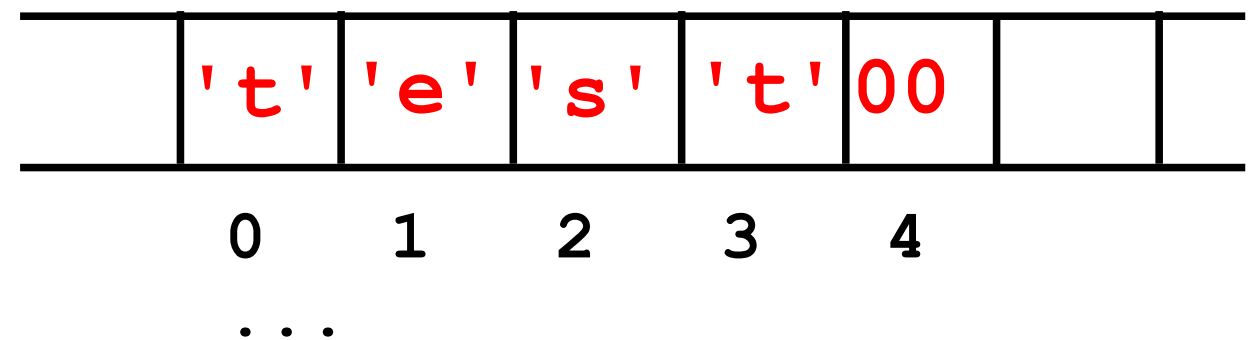


Definition von Character-Feldern („String-Variablen“)

```
char xy[10];           char xy[] = "test";  
...  
strcpy(&xy[0], "test");
```

Felder: Variablen gleichen Typs können über einen Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden.

```
xy[0] = 't';  
xy[1] = 'e';  
xy[2] = 's';  
xy[3] = 't';  
xy[4] = 0x00;
```

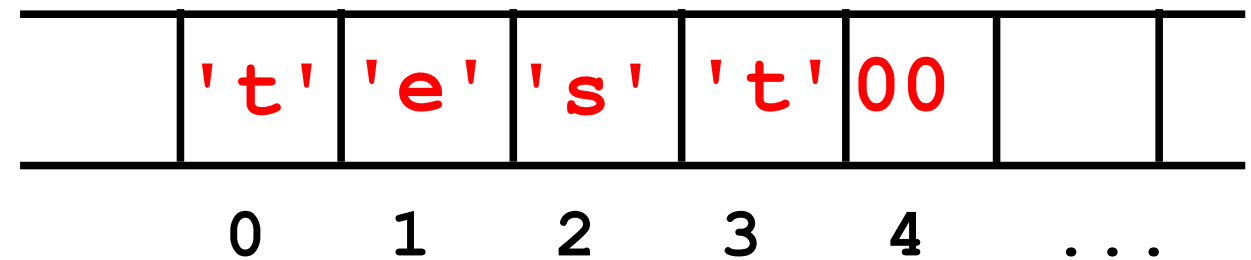


Definition von Character-Feldern („String-Variablen“)

```
char xy[10];           char xy[] = "test";  
...  
strcpy(&xy[0], "test");
```

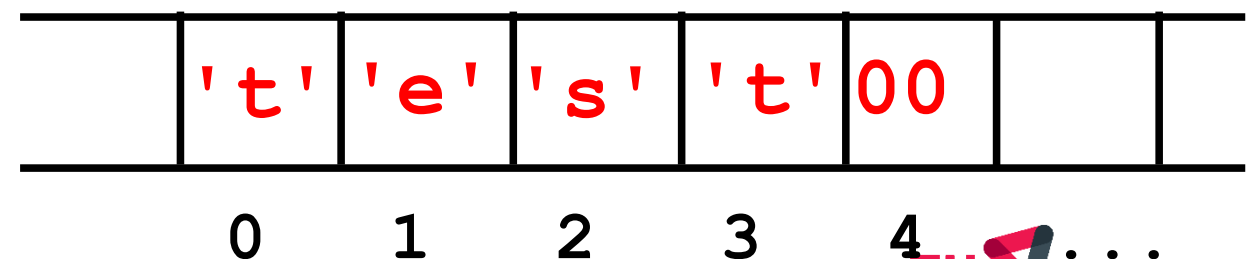
Felder: Variablen gleichen Typs können über einen Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden.

```
xy[0] = 't';  
xy[1] = 'e';  
xy[2] = 's';  
xy[3] = 't';  
xy[4] = 0x00;
```



```
strcpy(&xy[0], "test");
```

```
xy[i] = feld[i];  
if(feld[i] == 0x00) ...
```



Definition von Character-Feldern („String-Variablen“)

```
char pe_st[10];          char pe_st[] = "test";  
...  
strcpy(&pe_st[0], "test");
```

Ausgabe-Formatelement für Character-Felder (String-Variablen)

%	
-	(optional) linksbündig
b	(optional) Minimalbreite des Ausgabefeldes
.n	(optional) Breite des vom String beschriebenen Teil-Ausgabefeldes
s	char-array

```
printf ("String:%s\n\r", &pe_st[0]);  
printf ("% .3s\n\r", &pe_st[0]);
```

String:test
tes

Definition von Character-Feldern („String-Variablen“)

```
char pe_st[10];          char pe_st[] = "test";  
...  
strcpy(&pe_st[0], "test");
```

Ausgabe-Formatelement für Character-Felder (String-Variablen)

%	
-	(optional) linksbündig
<i>b</i>	(optional) Minimalbreite des Ausgabefeldes
<i>.n</i>	(optional) Breite des vom String beschriebenen Teil-Ausgabefeldes
s	char-array

```
printf ("String:%s\n\r", &pe_st[0]);
```

```
printf ("% .3s\n\r", &pe_st[0]);
```

String:test

tes

Definition von Character-Feldern („String-Variablen“)

```
char pe_st[10];          char pe_st[] = "test";  
...  
strcpy(&pe_st[0], "test");
```

Ausgabe-Formatelement für Character-Felder (String-Variablen)

%

-	(optional) linksbündig
<i>b</i>	(optional) Minimalbreite des Ausgabefeldes
<i>.n</i>	(optional) Breite des vom String beschriebenen Teil-Ausgabefeldes
<i>s</i>	char-array

```
printf ("String:%s\n\r", &pe_st[0]);
```

```
printf ("% .3s\n\r", &pe_st[0]);
```

String:test

tes

Erzeugung von Variablen

Einfaches Programmbeispiel

```
#include <stdio.h>
```

Bibliotheksfunktionen

```
// --- constants -----
```

```
#define VALUE 5
```

Definition von Konstanten

```
// --- prototypes -----
```

```
void main(void);
```

Bekanntgabe von Funktionen und Variablentypen

```
typedef unsigned char Byte;
```

```
// --- modul-variables -----
```

```
int count;
```

Erzeugen von Modul-Variablen
(Speicherplatz bereitstellen)

```
Byte out;
```

```
// --- start of program -----
```

```
void main(void)
```

„Hauptprogramm“ (erste Funktion)

```
{
```

```
    int state;
```

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)

```
    ...
```

```
    for (;;) 
```

Endlos-Schleife (kein Betriebssystem)

```
    {
```

```
        ...
```

```
    }
```

```
}
```

```
// --- end of main -----
```


Einfaches Programmbeispiel

```
#include <stdio.h>
// --- constants -----
#define VALUE 5
// --- prototypes -----
void main(void);
typedef unsigned char Byte;
// --- modul-variables -----
int count;
Byte out;
// --- start of program -----
void main(void)
{
    int state;
    ...
    for (;;)
    {
        ...
    }
}
// --- end of main -----
```

Bibliotheksfunktionen

Definition von Konstanten

Bekanntgabe von Funktionen und Variablentypen

Erzeugen von Modul-Variablen
(Speicherplatz bereitstellen)

„Hauptprogramm“ (erste Funktion)

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)

Endlos-Schleife (kein Betriebssystem)

Einfaches Programmbeispiel

<code>#include <stdio.h></code>	Bibliotheksfunktionen
<code>// --- constants -----</code>	
<code>#define VALUE 5</code>	Definition von Konstanten
<code>// --- prototypes -----</code>	
<code>void main(void);</code>	Bekanntgabe von Funktionen und Variablentypen
<code>typedef unsigned char Byte;</code>	
<code>// --- modul-variables -----</code>	
<code>int count;</code>	Erzeugen von Modul-Variablen
<code>Byte out;</code>	(Speicherplatz bereitstellen)
<code>// --- start of program -----</code>	
<code>void main(void)</code>	„Hauptprogramm“ (erste Funktion)
<code>{</code>	
<code> int state;</code>	Erzeugen von lokalen Variablen
<code> ...</code>	(Speicherplatz bereitstellen)
<code> for (;;) </code>	
<code> {</code>	Endlos-Schleife (kein Betriebssystem)
<code> ...</code>	
<code> }</code>	
<code>}</code>	
<code>// --- end of main -----</code>	

Einfaches Programmbeispiel

```
#include <stdio.h>
// --- constants -----
#define VALUE 5
// --- prototypes -----
void main(void);
typedef unsigned char Byte;
// --- modul-variables -----
int count;
Byte out;
// --- start of program -----
void main(void)
{
    int state;
    ...
    for (;;)
    {
        ...
    }
}
// --- end of main -----
```

Erzeugen von Modul-Variablen
(Speicherplatz bereitstellen)

Auf Modulvariablen kann von jeder
Stelle der Übersetzungseinheit zugegriffen
werden (aus dem „Hauptprogramm“ und
allen Unterprogrammen).
Definition vor der ersten Funktion.

Einfaches Programmbeispiel

```
#include <stdio.h>
// --- prototypes -----
void main(void);
typedef unsigned char Byte;
// --- modul-variables -----
int count;
Byte out;
// --- start of program -----
void main(void)
{
    int state;
    ...
    for (;;)
    {
        short int analog;
        ...
        ...
    }
}
```

Bibliotheksfunktionen

Bekanntgabe von Funktionen und Variablentypen

Erzeugen von Modul-Variablen
(Speicherplatz bereitstellen)

„Hauptprogramm“ (erste Funktion)

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)

Endlos-Schleife (kein Betriebssystem)

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)

Einfaches Programmbeispiel

```
#include <stdio.h>
// --- prototypes -----
void main(void);
typedef unsigned char Byte;
// --- modul-variables -----
int count;
Byte out;
// --- start of program -----
void main(void)
```

```
{
    int state;
    ...
    for (;;)
    {
        short int analog;
        ...
        ...
    }
}
```

Auf lokale Variablen kann nur in dem Block zugegriffen werden, in dem sie definiert sind.

Definition am Anfang des Blockes.

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)

Einfaches Programmbeispiel

```
#include <stdio.h>
// --- prototypes -----
void main(void);
typedef unsigned char Byte;
// --- modul-variables -----
int count;
Byte out;
// --- start of program -----
void main(void)
{
    int state;
    ...
    for (;;)
    {
        short int analog;
        ...
        ...
    }
}
```

Auf lokale Variablen kann nur in dem Block zugegriffen werden, in dem sie definiert sind.
Definition am Anfang des Blockes.

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)

Erzeugen von lokalen Variablen
(Speicherplatz bereitstellen)